

NetworkMatch App Summary

Based only on repository evidence in README, package manifests, source files, and schema as of 2026-04-06.

What It Is

NetworkMatch is a full-stack networking-event app with a host-facing admin dashboard and a mobile-first attendee experience. It creates timed matching rounds, pushes pairings in real time, and helps attendees save follow-ups after each conversation.

Who It's For

- Primary persona: event hosts or community organizers running structured in-person networking sessions; the attendee app supports guests joining from their phones during the event.

What It Does

- Lets admins register or log in, create events, set round length, and manage events from a dashboard.
- Generates a QR code and join link for each event so attendees can enter the mobile flow quickly.
- Lets attendees join with profile details, optional password signup, returning-user login, or emailed OTP recovery.
- Runs real-time rounds over Socket.IO with synced timers, lobby state, participant counts, and per-user match delivery.
- Scores matches using looking-for vs offering overlap, shared interests, role/company similarity, avoids repeat pairs, and creates trios when attendance is odd.
- Allows attendees to save connections and generate a follow-up LinkedIn message from a server-side template.
- Tracks badge progress and connection history across events for signed-in attendees.

How It Works

- Frontend apps: ``admin/`` and ``user/`` are separate Next.js 14 apps. The admin app handles event creation, QR display, participant lists, round controls, and event history. The user app handles join/auth, lobby, live match screens, saved connections, and badges.
- Backend: ``backend/src/index.js`` starts an Express API and Socket.IO server on the same HTTP server. REST routes are split into ``routes/admin.js``, ``routes/events.js``, and ``routes/users.js``, with JWT middleware in ``middleware/auth.js``.
- Data layer: ``backend/src/db/index.js`` uses ``pg`` to talk to PostgreSQL. ``backend/src/db/schema.sql`` defines admins, users, events, event_participants, matches, saved_connections, and user_badges.
- Real-time flow: clients store JWTs in localStorage, connect to Socket.IO with that token, join an ``event:{id}`` room, and receive ``event_state``, ``participant_count``, ``match_assigned``, ``timer_tick``, ``round_started``, ``round_ended``, and ``event_ended`` events.

- Matching flow: when an admin starts a round, the server selects active participants, runs `runMatchingAlgorithm`, inserts match rows, updates the event status/current round, emits round state, and starts an in-memory countdown timer.
- Supporting services: QR codes come from the `qrcode` package, OTP emails are sent through Resend, and LinkedIn follow-up text is generated by local template logic rather than an external AI API.
- Ephemeral state: active round timers, connected socket metadata, and shared user locations are held in server memory and are not persisted in PostgreSQL.

How To Run

- 1. Create a PostgreSQL database named `networkmatch` or update the connection string to your preferred database.
- 2. Copy `backend/.env.example` to `backend/.env` and set `DATABASE_URL`, `JWT_SECRET`, `ADMIN_INVITE_CODE`, `CLIENT_ADMIN_URL`, and `CLIENT_USER_URL`.
- 3. Copy `admin/.env.example` to `admin/.env.local` and `user/.env.example` to `user/.env.local`, keeping both `NEXT_PUBLIC_API_URL` and `NEXT_PUBLIC_WS_URL` pointed at the backend.
- 4. From the repo root, run `npm run install:all`, then `npm run db:init`.
- 5. Start everything with `npm run dev`. The repo config expects the backend on `http://localhost:4000`, the admin app on `http://localhost:3001`, and the user app on `http://localhost:3000`.
- 6. Open the admin app, create an event, then use the QR code or join URL to enter the attendee flow.